

Vježba 5 – Izuzeci

Izuzeci i prekidi

Izuzeci i prekidi su, pored grananja i skokova, događaji koji mijenjaju normalni tok izvršavanja instrukcija. Oni su prvobitno stvoreni da bi se nosili s neočekivanim događajima unutar procesora, poput aritmetičkog prekoračenja. Jedan su od komplikovanijih dijelova dizajna upravljačke strukture.

Mnoge arhitekture ne razlikuju prekide (**interrupts**) i izuzetke (**exceptions**), često koristeći stariji naziv prekid (interrupt) za oba tipa događaja. Na primjer, Intel x86 koristi prekide. Po MIPS konvenciji, koristećemo termin izuzetak (exception) za bilo koju neočekivanu promjenu toka kontrole bez razlikovanja da li je uzrok unutrašnji ili vanjski; a termin prekid (interrupt) samo kada je događaj uzrokovan vanjski. Primjeri izuzetka/prekida:

Type of event	From where?	MIPS terminology
I/O device request	External	Interrupt
Invoke the operating system from user program	Internal	Exception
Arithmetic overflow	Internal	Exception
Using an undefined instruction	Internal	Exception
Hardware malfunctions	Either	Exception or interrupt

Izuzeci u MIPS arhitekturi

Osnovna radnja koju procesor mora izvršiti kada se dogodi izuzetak je da spasi adresu instrukcije koja je dovela do izuzetka u **exception program counter (EPC)** i preusmjeri kontrolu na operativni sistem na određenu adresu (neka je ta ulazna tačka(adresa) **8000 0180**). Operativni sistem tada može izvršiti odgovarajuću akciju, što može uključivati pružanje neke usluge korisničkom programu, preduzimanje neke unaprijed definisane akcije u odgovoru na prekoračenje ili zaustavljanje izvršavanja programa i prijavu greške. Nakon obavljanja potrebnih radnji zbog izuzetka, operativni sistem može završiti izvršavanje programa ili nastaviti izvršavanje koristeći EPC kako bi odredio gdje nastaviti izvršavanje programa.

Da bi operativni sistem obradio izuzetak, pored instrukcije koja ga je uzrokovala, mora znati i razlog za izuzetak. Postoje dvije glavne metode za komunikaciju razloga za izuzetak. Metoda koja se koristi u MIPS arhitekturi je uključivanje registra statusa (**cause register**), koji sadrži polje koje ukazuje na razlog izuzetka.

Morat ćemo dodati još dva registra u našu trenutnu MIPS implementaciju:

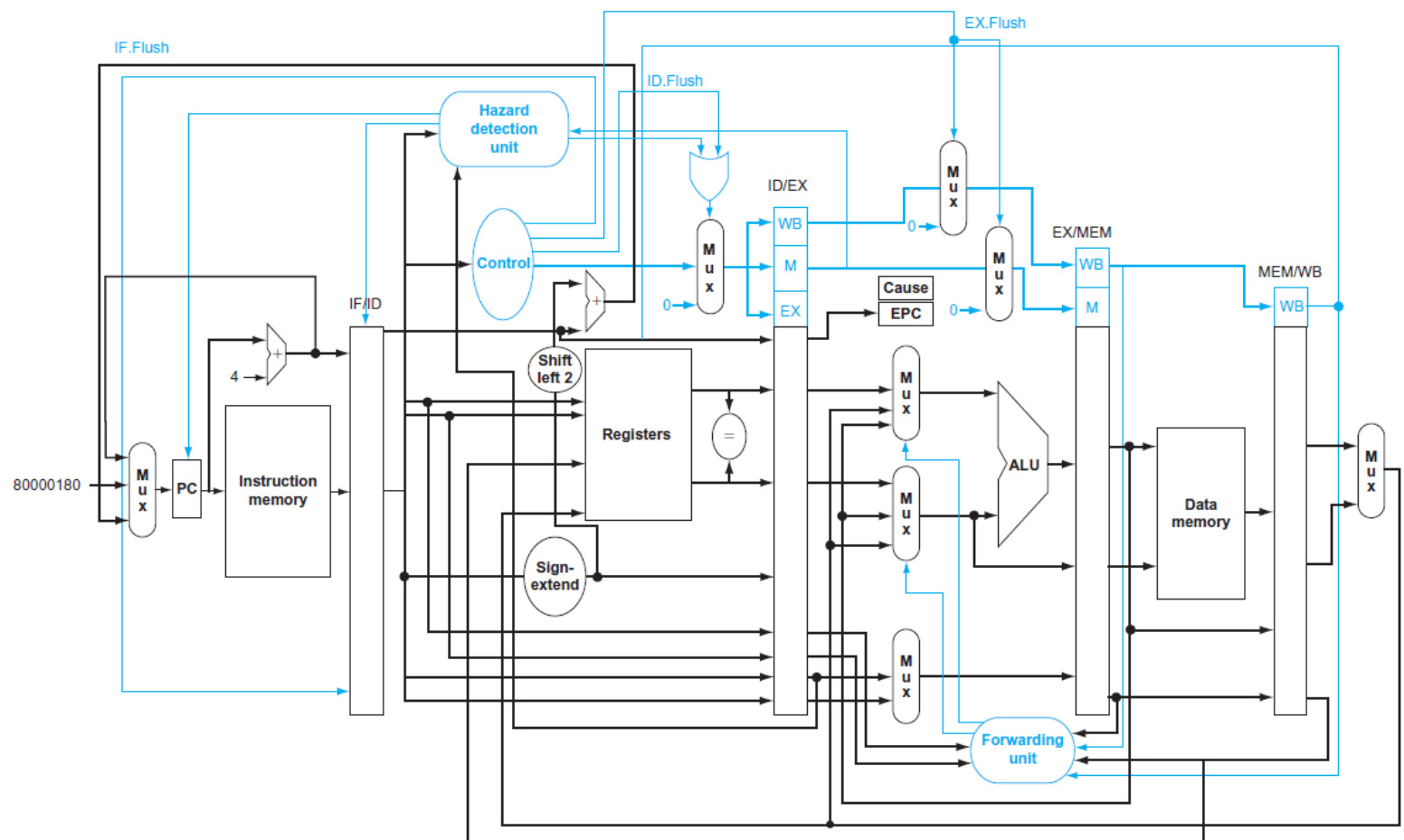
1. EPC: 32-bitni registar koji se koristi za čuvanje adrese pogođene instrukcije.
2. Cause: Registar koji se koristi za bilježenje uzroka izuzetka.

U MIPS protočnoj strukturi izuzeci se tretiraju kao jedan oblik kontrolnih hazarda. Na primjer, pretpostavimo da se javi aritmetičko prekoračenje (**arithmetic overflow**) u instrukciji sabiranja. Potrebno je isprazniti instrukcije koje dolaze nakon instrukcije sabiranja iz pipeline-a i početi preuzimati instrukcije sa nove adrese (sa adrese 8000 0180 – koja je ulazna tačka za obradu prekida).

Koristi se isti mehanizam kao i kod promašaja pri grananju. Kada smo se bavili promašajem u grananju, vidjeli smo kako isprazniti instrukciju u IF fazi tako što je pretvorimo u nop instrukciju. Za flush instrukcije u ID fazi, koristi se multiplexer koji je već u ID fazi, a koji postavlja kontrolne signale u nulu, kojim upravlja signal ID.Flush, ORuje se sa signalom za zastoje iz jedinice za otkrivanje hazarda. Za flush instrukcije u EX fazi, koristi se signal EX.Flush i odgovarajući multiplexer. Za početak preuzimanja instrukcija sa adrese 8000 0180, dodan je dodatni ulaz u PC multiplexer koji šalje 8000 0180 u PC.

Neki izuzeci zahtijevaju da se odbaci upis u određeni registar. Pošto se aritmetičko prekoračenje otkriva tokom EX faze moguće je koristiti EX.Flush signal da se spriječi upis rezultata instrukcije u WB fazi. Mnogi izuzeci zahtijevaju da se završi instrukcija koja je izazvala izuzetak kao da je normalno izvršena. Najlakši način za to je da uradimo flush instrukcije i ponovo je pokrenemo od početka nakon što se izuzetak obradi.

Zadnji korak je spremanje adrese instrukcije koja je dovela do prekoračenja u brojač programa izuzetka (EPC). U stvarnosti, sprema se **adresa+4** (pošto je ta vrijednost u pregradnom registru), tako da handler izuzetaka/prekida prvo mora oduzeti 4 od sačuvane vrijednosti.



Zadatak 1:

Neka je data sekvenca instrukcija ispod. Ukoliko se javi aritmetičko prekoračenje u četvrtoj instrukciji, prikazati stanje protočne strukture u ciklusima 6 i 7.

0x40: sub r11, r2, r4

0x44: and r12, r2, r5

0x48: or r13, r2, r6

0x4C: add r1, r2, r1 //izuzetak u EXE fazi ove instrukcije

0x50: slt r15, r6, r7

0x54: lw r16, 50(r7)

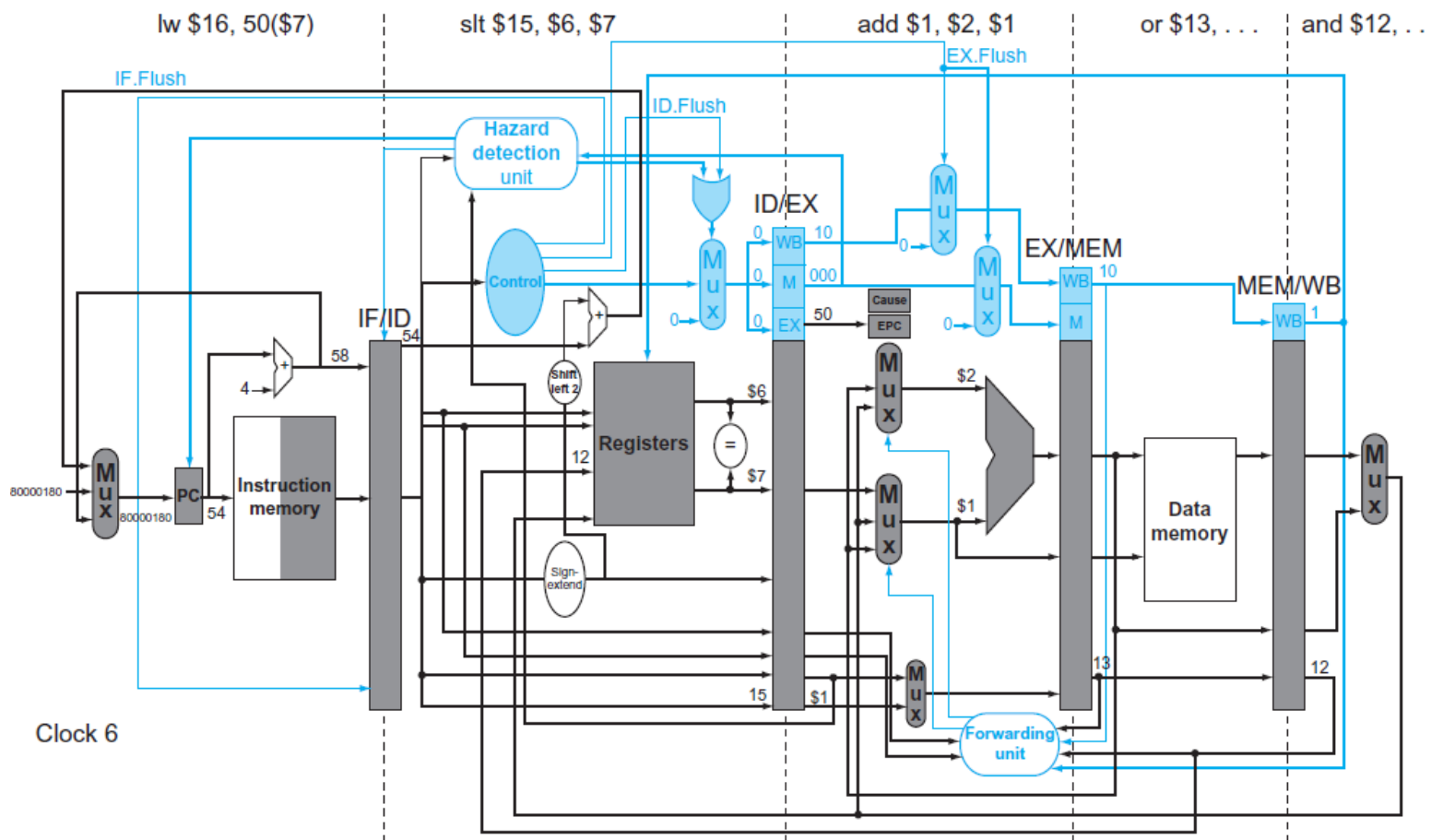
Neka su prve dvije instrukcije na adresi 8000 0180 sljedeće (ovo su prve dvije instrukcije koda koji radi obradu prekida i izuzetaka):

0x80000180: sw \$26, 1000(\$0)

0x80000184: sw \$27, 1004(\$0)

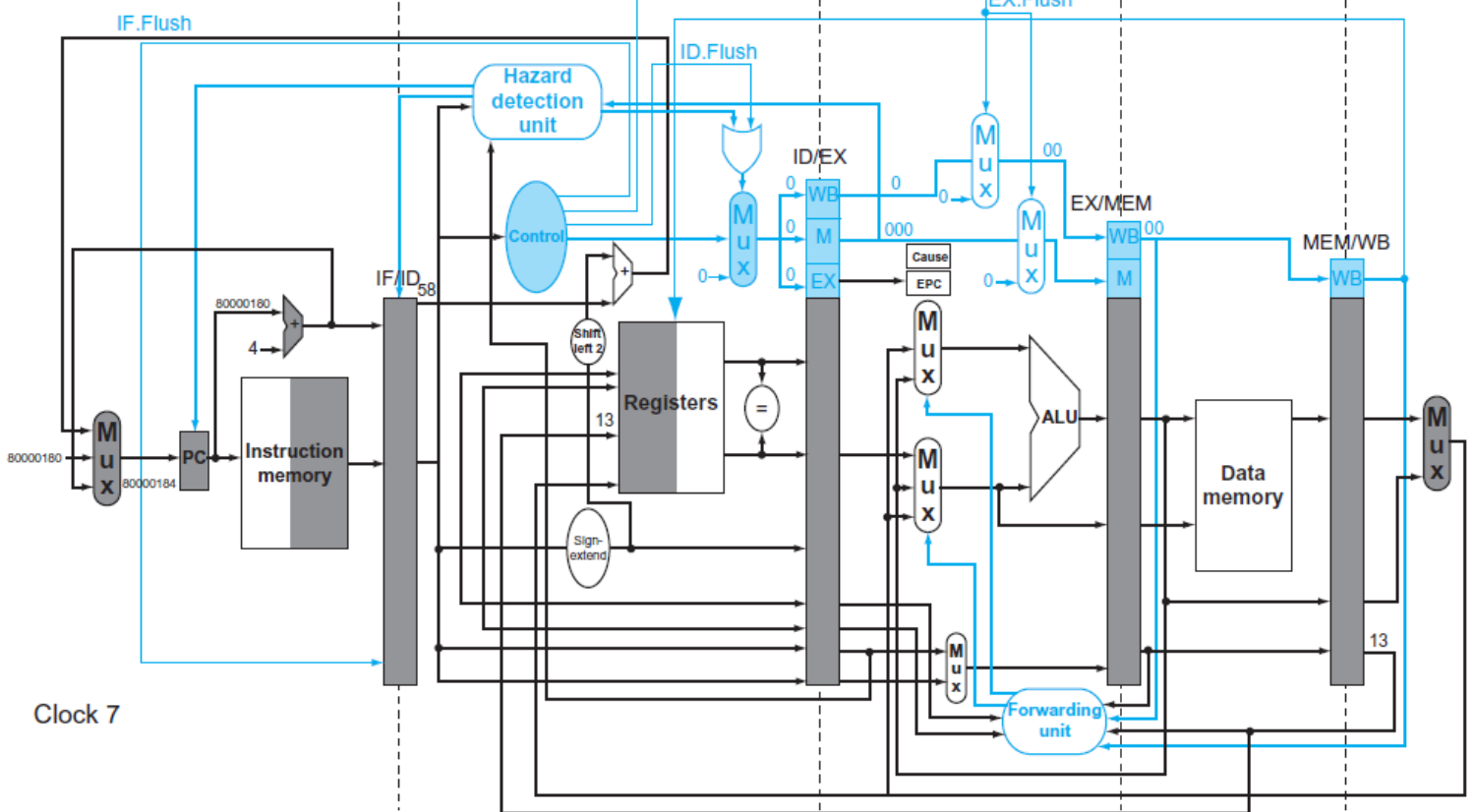
Izuzetak je detektovan u EXE fazi add instrukcije u ciklusu 6. U EPC registar se snima adresa sljedeće instrukcije (PC+4 vrijednost za add instrukciju = 0x50).

Pored toga, izuzetak uzrokuje da se vrijednosti svih flush signala (IF.Flush, ID.Flush, EX.Flush) postave na 1 pri kraju ciklusa 6 čime se add instrukcija i dvije instrukcije poslije add (slt – koja je u svojoj ID fazi, i lw – koja je u IF fazi u ciklusu 6) izbacuju iz protočne strukture.



U ciklusu 7 add, slt i lw instrukcije su pretvorene u nop instrukcije. Pored toga, počinje se sa izvršenjem prve instrukcije sa adrese 8000 0180. Dalje, operativni sistem obraditi izuzetak.

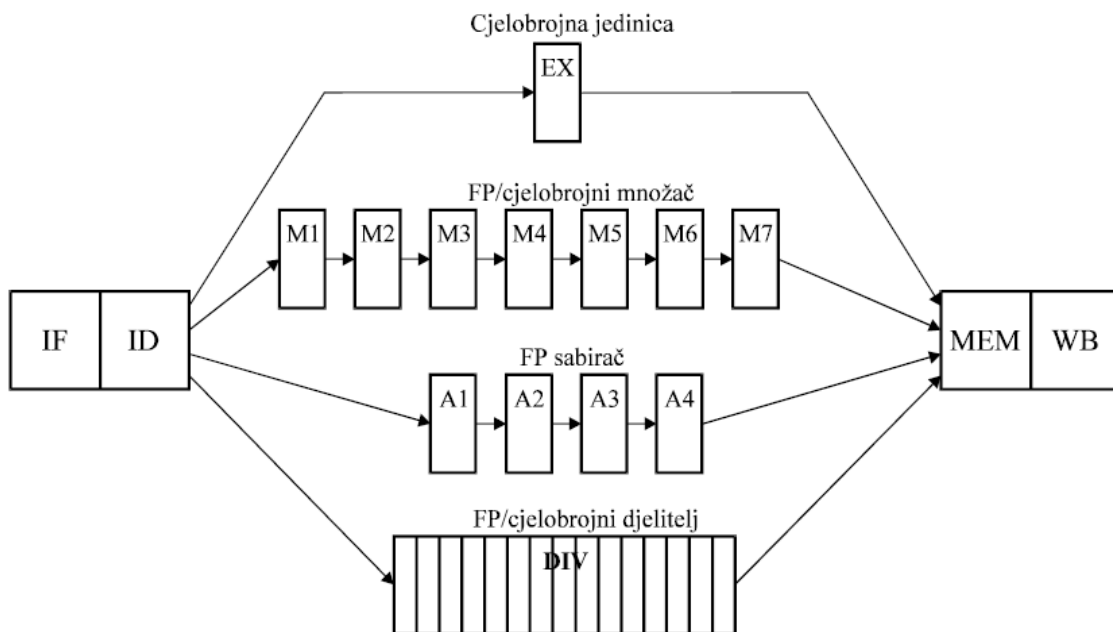
sw \$26, 1000(\$0) bubble (nop) bubble bubble or \$13, ...



Višeciklusne instrukcije

Različite instrukcije imaju i različita vremena izvršavanja. Izvršavanje FP-operacija (floating-point) je vremenski zahtjevnije nego izvršavanje cjelobrojnih operacija. Zahtijevati izvršenje neke FP-operacije u jednom ciklusu sata nije praktično, s obzirom da bi to dodatno zakomplikovalo hardver te bi (značajno) produžilo trajanje jednog ciklusa sata (ne treba zaboraviti na činjenicu da je u protočnoj strukturi trajanje ciklusa određeno trajanjem najdužeg ciklusa).

Jedno od rješenja je da se EX faza može ponavljati više puta, te da postoji više odvojenih FP-funkcionalnih jedinica koje zahtijevaju različita vremena izvršenja, pri čemu te funkcionalne jedinice mogu (ali ne moraju) biti protočne i mogu dopustiti nekoj instrukciji da započne (svoju) fazu izvršavanja prije nego je prethodna instrukcija (koja se duže izvršava) završila (svoju) fazu izvršavanja. S obzirom da faza izvršenja sada traje više ciklusa, ovakve operacije se zovu višeciklusnim. Na slici je prikazana protočna struktura koja podržava više istovrmenih FP operacija.



Naprimjer, neka se izvršava sljedeća sekvenca instrukcija, uz pretpostavku da je za izvršavanje **mul.d** instrukcije potrebno 7 ciklusa:

```
mul.d f4, f1, f2 – I1
add r3, r1, r2 – I2
sub r7, r2, r1 – I3
and r8, r1, r2 – I4
mul.d f2, f5, f6 – I5
add r9, r5, r6 – I6
```

Prikaz faza izvršenja ovih instrukcija po ciklusima je prikazan kroz sljedeću tabelu:

	C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14	C15
I1	IF	ID	Ex1	Ex2	Ex3	Ex4	Ex5	Ex6	Ex7	M	W				
I2		IF	ID	EX	M	W									
I3			IF	ID	EX	M	W								
I4				IF	ID	EX	M	W							
I5					IF	ID	Ex1	Ex2	Ex3	Ex4	Ex5	Ex6	Ex7	M	W
I6						IF	ID	EX	M	W					

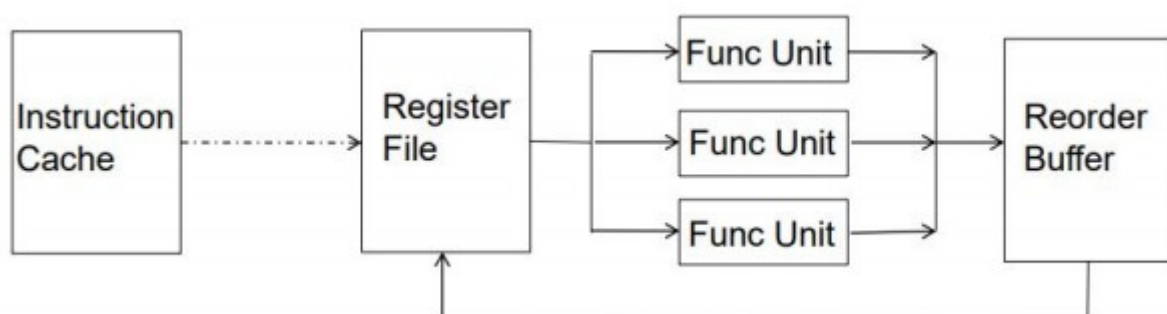
Iz tabele se može vidjeti da izvršenje instrukcije **mul.d** (koja predstavlja FP-instrukciju) traje znatno duže nego izvršenje ostalih instrukcija (koje su cjelobrojne). Jasno je da ovakav način izvršavanja ne podržava sekvencijalno izvršenje instrukcija, što može da izazove određene probleme. Postavlja se pitanje šta će se desiti ako izvršavanje **mul.d** instrukcije dovede do izuzetka (nastaje tzv. neprecizni izuzetak - stanja nekih od registara nepovratno promijenjena), s obzirom da bi stanje arhitekture trebalo biti konzistentno, u trenutku kada je izuzetak spreman za obradu, pri čemu konzistentnost stanja arhitekture podrazumijeva dvije stvari:

1. Sve instrukcije prije instrukcija koja je izazvala izuzetak moraju biti izvršene.
2. Nijedna instrukcija nakon instrukcije koja je izazvala izuzetak ne smije biti izvršena.

Jedan od načina rješavanja ovog problema je baferovanje rezultata dok se ranije pokrenute instrukcije ne izvrše. Na ovaj način se postiže da se instrukcije izvršavaju van reda (**out of order-OoO**), ali se prije upisa rezultata (prije promjene trenutnog stanja arhitekture) ponovo redaju u prvobitni redoslijed, te tim redom upisuju rezultate svog izvršavanja. Ova tehnika je, zbog tog ponovnog redanja pri upisu rezultata, poznata i pod nazivom **reorder buffer**, s obzirom da se to ponovno redanje obavlja na temelju unosa u bafer. Dakle, procedura je sljedeća:

1. Nakon što se instrukcija dekodira, ona rezerviše mjesto u baferu.
2. Nakon što završi svoje izvršavanje, rezultat izvršavanja se upisuje na rezervisano mjesto u baferu.
3. Nakon što se instrukcija sa najstarijim unosom u baferu izvrši bez izuzetaka, rezultat se iz bafera prepisuje u memoriju ili u neki od registara, u zavisnosti od instrukcija.

Grafički, ova tehnika se može prikazati na sljedeći način:



Korištenjem ove tehnike, prikaz faza izvršenja instrukcija po ciklusima je sljedeći (pri čemu oznaka R govori da instrukcija rezultat piše u bafer, dok oznaka W podrazumijeva upis rezultata u registar):

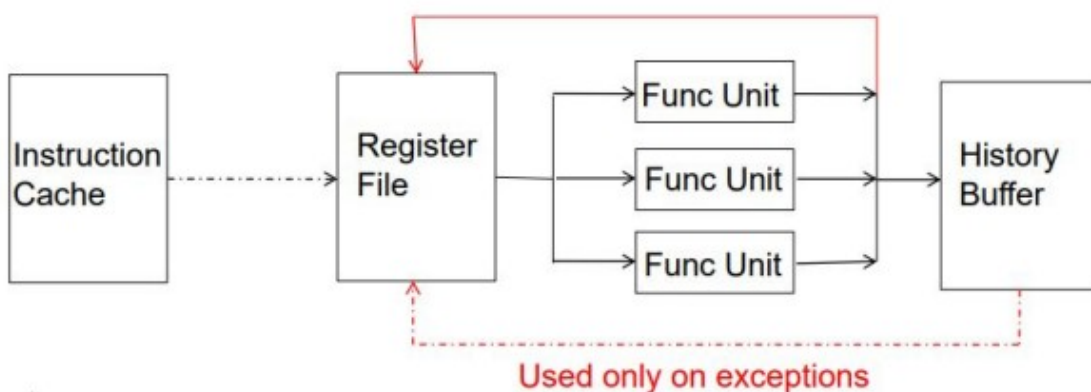
	C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14	C15	C16	C17
I1	IF	ID	Ex1	Ex2	Ex3	Ex4	Ex5	Ex6	Ex7	M	R	W					
I2		IF	ID	EX	M	R							W				
I3			IF	ID	EX	M	R							W			
I4				IF	ID	EX	M	R							W		
I5					IF	ID	Ex1	Ex2	Ex3	Ex4	Ex5	Ex6	Ex7	M	R	W	
I6						IF	ID	EX	M	R							W

Ukoliko je instrukciji koja se trenutno izvršava potreban rezultat prethodne instrukcije (koja nije završila svoje izvršenje), potrebno je ili uvesti zastoj dok prethodna instrukcija ne upiše rezultat svog izvršavanja ili proslijediti potrebnu vrijednost, pri čemu se vrijednost koju je potrebno proslijediti može nalaziti ili u baferu rezultata (reorder baferu) ili u (pregradnim) registrima. Iz ovog se može zaključiti da je povećana i (hardverska i vremenska) kompleksnost prosljeđivanja, što predstavlja nedostatak ove tehnike. Još jedan od nedostataka ove tehnike proizilazi iz činjenice da ukoliko su vremena izvršenja instrukcija značajno različita, povećava se broj baferovanih rezultata, što dalje znači da je potreban veći broj (hardverskih) komponenti, veći MUX-evi itd.

Još jedan od načina rješavanja problema izuzetaka je korištenjem zapisa o historiji događaja (**history buffer**). Osnovna ideja ove tehnike je da se rezultat upisuje u registre kada instrukcija završi svoje izvršavanje, odnosno, instrukcija mijenja trenutno stanje arhitekture, ali ukoliko dođe do izuzetka ta promjena se poništava. Da bi se instrukcija mogla poništiti, potrebno je negdje zapisati vrijednost registra prije promjene, pa se za tu svrhu koristi **history buffer**. Procedura je sljedeća:

1. Nakon što se instrukcija dekodira, ona rezerviše mjesto u baferu.
2. Nakon što instrukcija završi svoje izvršenje, ona pohranjuje staru vrijednost odredišnog registra u rezervisano mjesto u baferu.
3. Ako je najstarija instrukcija u **history buffer-u** završila svoje izvršavanje bez izuzetaka, rezervisano mjesto u baferu se oslobađa.
4. Ako se prilikom izvršavanja najstarije instrukcije u **history buffer-u** desi izuzetak, potrebno je poništiti sve promjene trenutnog stanja arhitekture koje su napravile instrukcije koje su svoje izvršavanje započele nakon te instrukcije, korištenjem zapisa o starim vrijednostima u **history buffer-u**.

Grafički, ova tehnika se može prikazati na sljedeći način:



Glavni nedostatak ove tehnike je što je potrebno čitati stare vrijednosti registara, u slučaju izuzetka, čime je povećana vremenska kompleksnost obrade izuzetka (za razliku od *reorder buffer* tehnike, gdje je povećana kompleksnost pristupa vrijednostima registara).

Pored opisanog problema sa izuzecima, višeciklusne operacije (kao i “jednociklusne”) mogu da izazovu **hazarde podataka**. Pored **RAW** (*read after write*) hazarda, kada trenutna instrukcija mora čekati da neka ranije pokrenuta instrukcija upiše rezultat svog izvršavanja prije nego što trenutna instrukcija pročitati taj rezultat, višeciklusne operacije mogu da izazovu i **WAW** (*write after write*) hazarde, kada instrukcija koja se trenutno izvršava mora čekati da neka ranije pokrenuta upiše rezultat svog izvršavanja prije nego što trenutna instrukcija upiše rezultat svog izvršavanja.

Naprimjer, neka se izvršava sljedeća sekvenca instrukcija:

```
add r3, r1, r2 – I1
sub r7, r2, r1 – I2
add.d f8, f1, f2 – I3
add r3, r1, r2 – I4
l.d f8, 0(r5) – I5
```

Prikaz faza izvršenja ovih instrukcija po ciklusima je:

	C1	C2	C3	C4	C5	C6	C7	C8	C9	C10
I1	IF	ID	EX	M	W					
I2		IF	ID	EX	M	W				
I3			IF	ID	Ex1	Ex2	Ex3	Ex4	M	W
I4				IF	ID	EX	M	W		
I5					IF	ID	EX	M	W	

Vidimo da postoji **WAW** hazard, **I.d** instrukcija upisuje rezultat u registar **f8** prije instrukcije **add.d**. **WAW** hazard se rješava na dva načina:

1. Zadržati **I.d** dok **add.d** ne uđe u MEM segment
2. “Suzbiti” rezultat **add.d** otkrivanjem hazarda i promjenom upravljanja, tako da **add.d** ne upiše svoj rezultat (korištenjem reorder buffer-a). Tada se **I.d** može pokrenuti odmah.

Pri otkrivanju mogućih hazarda treba razmotriti iste između FP-instrukcija kao i FP i cjelobrojnih instrukcija. Sa izuzetkom **FP-load/store** i **FP-int register move**, cjelobrojne instrukcije rade samo sa cjelobrojnim registrima, dok FP-instrukcije rade samo sa FP-skupom registara. Zato treba samo paziti na te instrukcije pri detekciji hazarda između FP i cjelobrojnih instrukcija. Pod pretpostavkom da PS otkriva hazarde u ID-segmentu, postoje tri provjere prije pokretanja instrukcije:

1. Provjera strukturnog hazarda - treba sačekati da tražena funkcionalna jedinica prestane biti zauzeta i obezbijediti da je port za upis u registre slobodan kada bude potreban.
2. Provjera RAW hazarda podataka - čekati sve dok izvorišni registri više ne budu na spisku odredišta na čekanju (*pending destinations*).
3. Provjera na WAW hazard podataka - provjeriti da li ijedna instrukcija u A1, ..., A4, D, M1, ..., M7, ima isti odredišni registar kao tekuća, ako ima, uvesti zastoje u pokretanju te instrukcije (u ID-segmentu).

Zadatak 2.

Odrediti broj ciklusa potrebnih za izvršenje sekvence instrukcija u oglednoj protočnoj strukturi

- bez prosljeđivanja.
- sa prosljeđivanjem.

Prikazati u kojoj fazi se nalazi svaka instrukcija u određenom ciklusu. Pretpostaviti da se za rješavanje problema sa izuzecima ne koristi nijedna od ranije opisanih tehnika (problem se ignoriše). Neka su na raspolaganju četiri funkcionalne (množac i sabirač su protočni, djelitelj nije).

l.d f0, 16(r0) – I1

l.d f1, 64(r0) – I2

mul.d f4, f1, f0 – I3

add.d f4, f1, f2 – I4

sub.d f5, f1, f0 – I5

- Potrebno je 17 ciklusa, raspored po fazama je:

	C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14	C15	C16	C17
I1	IF	ID	Ex1	M	W												
I2		IF	ID	Ex1	M	W											
I3			IF	ID	—	—	Ex1	Ex2	Ex3	Ex4	Ex5	Ex6	Ex7	M	W		
I4				IF	—	—	ID	Ex1	Ex2	Ex3	Ex4	...	...	...	M	W	
I5							IF	ID	Ex1	Ex2	Ex3	...	...	...	Ex4	M	W

Zastoji označeni sa simbolom '—' su uzrokovani RAW hazardom, dok su zastoji označeni sa '...' uzrokovani WAW hazardom.

- Potrebno je 16 ciklusa.

	C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14	C15	C16	C17
I1	IF	ID	Ex1	M	W												
I2		IF	ID	Ex1	M	W											
I3			IF	ID	—	Ex1	Ex2	Ex3	Ex4	Ex5	Ex6	Ex7	M	W			
I4				IF	—	ID	Ex1	Ex2	Ex3	Ex4	...	...	...	M	W		
I5						IF	ID	Ex1	Ex2	Ex3	...	...	...	Ex4	M	W	

- Ponoviti podzadatak a), sa razlikom da sada nijedna funkcionalna jedinica nije protočna.

	C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14	C15	C16	C17	C18	C19	C20
I1	IF	ID	Ex1	M	W															
I2		IF	ID	Ex1	M	W														
I3			IF	ID	—	—	Ex1	Ex2	Ex3	Ex4	Ex5	Ex6	Ex7	M	W					
I4				IF	—	—	ID	Ex1	Ex2	Ex3	Ex4	...	...	...	M	W				
I5							IF	ID	***	***	***	...	...	...	Ex1	Ex2	Ex3	Ex4	M	W

Zastoji označeni sa simbolom '***' su zbog strukturnih hazarda, pošto je sabirač potreban posljednoj instrukciji u upotrebi do 15. ciklusa.

Zadatak 3.

Odrediti broj ciklusa potrebnih za izvršenje sekvence instrukcija u ogleđnoj protočnoj strukturi bez prosljeđivanja. Prikazati u kojoj fazi se nalazi svaka instrukcija u određenom ciklusu. Pretpostaviti da se za rješavanje problema sa izuzecima koristi **reorder buffer** tehnika. Neka su na raspolaganju četiri funkcionalne jedinice. Prikazati i šta se nalazi u baferu rezultata u svakom od ciklusa.

add.d f0, f1, f2 – I1

mul.d f1, f2, f3 – I2

sub.d f2, f3, f4 – I3

add.d f3, f4, f5 – I4

mul.d f4, f5, f6 – I5

U protočnoj strukturi bez prosljeđivanja potrebno je 16 ciklusa.

	C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14	C15	C16
I1	IF	ID	Ex1	Ex2	Ex3	Ex4	M	R	W							
I2		IF	ID	Ex1	Ex2	Ex3	Ex4	Ex5	Ex6	Ex7	M	R	W			
I3			IF	ID	Ex1	Ex2	Ex3	Ex4	M	R				W		
I4				IF	ID	Ex1	Ex2	Ex3	Ex4	M	R				W	
I5					IF	ID	Ex1	Ex2	Ex3	Ex4	Ex5	Ex6	Ex7	M	R	W

Izgled bafera u svakom ciklusu:

Ciklus	Instrukcija	Razlog pristupa	Odredišni registar
1			
2	I1	Rezervacija	
3	I2	Rezervacija	
4	I3	Rezervacija	
5	I4	Rezervacija	
6	I5	Rezervacija	
7			
8	I1	Upis rezultata	F0
9	I1	Čitanje vrijednosti	
10	I3	Upis rezultata	F2
11	I4	Upis rezultata	F3
12	I2	Upis rezultata	F1
13	I2	Čitanje vrijednosti	
14	I3	Čitanje vrijednosti	
15	I4	Čitanje vrijednosti	
	I5	Upis rezultata	F4
16	I5	Čitanje vrijednosti	

Baferima u kojima se čuvaju rezultati može se pristupiti ili zbog rezervacije (u ID fazi svake instrukcije) ili zbog upisa rezultata izvršavanja (nakon što se završi EX faza svake instrukcije) ili zbog čitanja vrijednosti (kada dođe vrijeme za upis rezultata instrukcije u odredišni registar - WB faza).

Iz tabele se može vidjeti da instrukcije upisuju rezultate izvršavanja u bafer različitim redoslijedom (od originalnog), ali se čitanje tih upisanih vrijednosti obavlja u ispravnom redoslijedu.

Zadatak 4.

Neka su vrijednosti registara takve da vrijedi $f_i = i$ i $r_i = i$. Na raspolaganju su četiri funkcionalne jedinice. Pretpostaviti da se za rješavanje problema sa izuzecima koristi **history buffer**. Koje vrijednosti će imati odredišni registri u sljedećoj sekvenci instrukcija ukoliko je došlo do izuzetka tokom izvršavanja prve instrukcije i to u 6. ciklusu EX faze? Prikazati i šta se nalazi u history buffer-u u svakom ciklusu.

```
mul.d f4, f1, f2 – I1
add r3, r6, r2 – I2
sub r7, r2, r1 – I3
add r7, r1, r2 – I4
mul.d f2, f5, f6 – I5
add r9, r5, r6 – I6
```

Za izvršenje ove sekvence instrukcija (uz pretpostavku da nijedna od instrukcija ne izazove izuzetak) potrebno je 15 ciklusa.

	C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14	C15
I1	IF	ID	Ex1	Ex2	Ex3	Ex4	Ex5	Ex6	Ex7	M	W				
I2		IF	ID	EX	M	W									
I3			IF	ID	EX	M	W								
I4				IF	ID	EX	M	W							
I5					IF	ID	Ex1	Ex2	Ex3	Ex4	Ex5	Ex6	Ex7	M	W
I6						IF	ID	EX	M	W					

History buffer u svakom ciklusu (spremaju se stare vrijednosti):

Ciklus	Instrukcija	Razlog pristupa	Odredišni registar
1			
2	I1	Rezervacija	
3	I2	Rezervacija	
4	I3	Rezervacija	
5	I4	Rezervacija	
6	I5 I2	Rezervacija Upis rezultata	R3 = 3
7	I6 I3	Rezervacija Upis rezultata	R7 = 7
8	I4	Upis rezultata	R7 = 1
9			
10	I6	Upis rezultata	R9 = 9
11	I1	Upis rezultata	F4 = 4.0
12			

13			
14			
15	I5	Upis rezultata	F2 = 2.0

U ovom slučaju upis rezultata podrazumijeva upis stare vrijednosti registra, kako bi se ista mogla vratiti u slučaju izuzetka.

Ukoliko dođe do izuzetka u 6. ciklusu EX faze prve instrukcije, odnosno u 8. ciklusu izvršavanja sekvence instrukcija, potrebno je osigurati konzistentno stanje arhitekture prije nego što se krene u obradu izuzetka. **Konzistentno stanje podrazumijeva da sve instrukcije prije te instrukcije trebaju biti izvršene, te da nijedna instrukcija nakon te instrukcije ne smije biti izvršena.** U 8. ciklusu svoje izvršavanje su završile 3 instrukcije nakon instrukcije koja je izazvala izuzetak (prva instrukcija). Sada je potrebno na osnovu rezultata zapisanih u history buffer-u poništiti izvršenje ovih instrukcija. Na temelju zapisa u ovom baferu, zaključujemo da je u 7. ciklusu registar R7 imao vrijednost 1, dok je u 6. ciklusu imao vrijednost 7 (pa je to konačna vrijednost za ovaj registar). U 5. ciklusu registar R3 je imao vrijednost 3 i to je konačna vrijednost za ovaj registar. Vrijednosti ostalih odredišnih registara nisu mijenjane. Nakon što su vraćene vrijednosti ovih registara (dobijeno je konzistentno stanje arhitekture), sve je spremno za obradu nastalog izuzetka.

Future file

Future file mehanizam koristi dvije registarske datoteke:

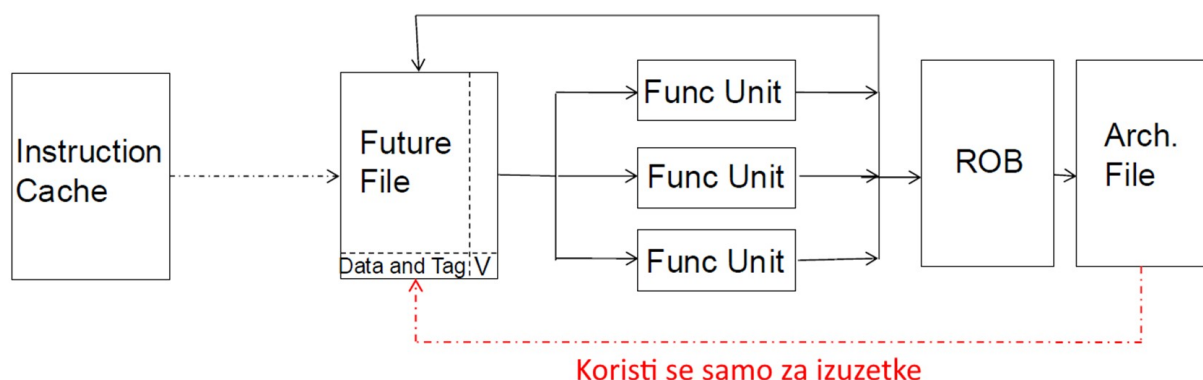
1. **Future file** predstavlja frontend registarsku datoteku i služi za brz pristup najnovijim vrijednostima registara u spekulativnom stanju. U nju se upisuju rezultati instrukcija koje su izračunate, ali još nisu konačno potvrđene.

2. **Arhitekturna registarska datoteka** predstavlja backend registarsku datoteku i čuva samo potvrđeno, ispravno stanje procesora, pa se koristi za oporavak u slučaju izuzetaka.

U R fazi rezultat instrukcije se upisuje u reorder buffer (ROB) i u future file, tako da naredne instrukcije mogu odmah koristiti najnoviju vrijednost.

U W fazi provjerava se da li postoji izuzetak:

- Ako izuzetka nema, rezultat se upisuje u arhitekturnu registarsku datoteku ili u memoriju.
- Ako izuzetak postoji, protočna struktura se prazni, arhitekturna registarska datoteka se kopira u future file, a izvršavanje se nastavlja od rutine za obradu izuzetka.



Zadatak 5.

Neka se sekvenca instrukcija izvršava u oglednoj protočnoj strukturi sa prosljeđivanjem. Neka su vrijednosti registara takve da vrijedi $f_i = i$. Neka je kašnjenje protočnog sabirača 4 ciklusa i protočnog množača 8 ciklusa. Pretpostaviti da se za rješavanje problema sa izuzecima koristi **future file**.

Napisati:

- Stanje future file-a (spekulativne registarske datoteke) u svakom ciklusu izvršenja sekvence
- Stanje arhitekturne registarske datoteke u svakom ciklusu izvršenja sekvence.
- Neka se desi izuzetak tokom izvršavanja treće instrukcije i to u 8. ciklusu EX faze. Šta će se desiti sa registarskim datotekama i reorder buffer-om?

add.d f4, f1, f2 – I1

sub.d f1, f4, f3 – I2

mul.d f2, f4, f5 – I3

add.d f5, f4, f3 – I4

sub.d f3, f2, f3 – I5

	C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12	C13	C14	C15	C16	C17	C18	C19	C20	C21	C22
I1	IF	ID	Ex 1	Ex 2	Ex 3	Ex 4	M	R	W													
I2		IF	ID	–	–	–	Ex 1	Ex 2	Ex 3	Ex 4	M	R	W									
I3			IF	–	–	–	ID	Ex 1	Ex 2	Ex 3	Ex 4	Ex 5	Ex 6	Ex 7	Ex 8	M	R	W				
I4							IF	ID	Ex 1	Ex 2	Ex 3	Ex 4	M	R					W			
I5								IF	ID	–	–	–	–	–	–	Ex 1	Ex 2	Ex 3	Ex 4	M	R	W

Odgovori:

- Upis u future file se desi u R fazi (kada i upis u reorder buffer).
- Upis u arhitekturnu registarsku datoteku se desi u W fazi.

Isječak relevantnih registara (f1 - f5) za svaki ciklus:

Ciklus	Future file					Arhitekturna reg. datoteka					Komentar
	F1	F2	F3	F4	F5	F1	F2	F3	F4	F5	
1	1.0	2.0	3.0	4.0	5.0	1.0	2.0	3.0	4.0	5.0	Početno stanje
2...7	1.0	2.0	3.0	4.0	5.0	1.0	2.0	3.0	4.0	5.0	Bez promjene (nijedna instrukcija nije bila u R ili W fazi)
8	1.0	2.0	3.0	3.0	5.0	1.0	2.0	3.0	4.0	5.0	I1: R faza
9	1.0	2.0	3.0	3.0	5.0	1.0	2.0	3.0	3.0	5.0	I1: W faza
10	1.0	2.0	3.0	3.0	5.0	1.0	2.0	3.0	3.0	5.0	
11	1.0	2.0	3.0	3.0	5.0	1.0	2.0	3.0	3.0	5.0	

12	0.0	2.0	3.0	3.0	5.0	1.0	2.0	3.0	3.0	5.0	I2: R faza
13	0.0	2.0	3.0	3.0	5.0	0.0	2.0	3.0	3.0	5.0	I2: W faza
14	0.0	2.0	3.0	3.0	6.0	0.0	2.0	3.0	3.0	5.0	I4: R faza
15	0.0	2.0	3.0	3.0	6.0	0.0	2.0	3.0	3.0	5.0	
16	0.0	2.0	3.0	3.0	6.0	0.0	2.0	3.0	3.0	5.0	
17	0.0	15.0	3.0	3.0	6.0	0.0	2.0	3.0	3.0	5.0	I3: R faza
18	0.0	15.0	3.0	3.0	6.0	0.0	15.0	3.0	3.0	5.0	I3: W faza
19	0.0	15.0	3.0	3.0	6.0	0.0	15.0	3.0	3.0	6.0	I4: W faza
20	0.0	15.0	3.0	3.0	6.0	0.0	15.0	3.0	3.0	6.0	
21	0.0	15.0	12.0	3.0	6.0	0.0	15.0	3.0	3.0	6.0	I5: R faza
22	0.0	15.0	12.0	3.0	6.0	0.0	15.0	12.0	3.0	6.0	I5: W faza

c) Ako se desi izuzetak u 8. ciklusu EX faze 3. instrukcije: u W fazi I3 (ciklus 18) će se prekopirati stanje arhitekturne reg. datoteke sa kraja 17. ciklusa u spekulativnu datoteku (future file). Time će rezultat I4 biti izbačen iz FF-a, čime se osigura konzistentno stanje arhitekture. Pored toga, sve instrukcije poslije I3 će biti izbačene iz protočne strukture (i iz reorder buffer-a).

Zadaci za samostalan rad

1. Neka se sljedeća sekvenca instrukcija izvršava u oglednoj MIPS arhitekturi sa prosljeđivanjem koja koristi *reorder buffer*:

- **add f1, f3, f5**
- **sub f3, f1, f2**
- **muld f3, f1, f5**
- **add f1, f2, f2**
- **muld f1, f3, f1**

Neka je kašnjenje množača 7 ciklusa i sabirača 4 ciklusa.

a) U kojem ciklusu se radi WB faza četvrte instrukcije?

b) Prikazati stanje reorder buffer-a u 7. ciklusu izvršenja sekvence.

c) Šta će se desiti ako 3. instrukcija izazove izuzetak u svojoj posljednjoj EX fazi?

2. Neka se sljedeća sekvenca instrukcija izvršava u oglednoj MIPS arhitekturi sa prosljeđivanjem koja koristi *reorder buffer*:

- **muld f1, f2, f3**
- **muld f3, f2, f1**
- **add f5, f2, f4**
- **subd f4, f2, f6**
- **add f6, f8, f2**

Neka je kašnjenje množača 7 ciklusa i sabirača 3 ciklusa. Koliko je ciklusa potrebno za izvršenje sekvence instrukcija?

3. Neka se sljedeća sekvenca instrukcija izvršava u oglednoj MIPS arhitekturi sa prosljeđivanjem koja koristi *history buffer*:

- **muld f1, f2, f3**
- **muld f3, f2, f1**
- **add f5, f2, f4**
- **subd f4, f2, f6**
- **add f6, f8, f2**

Neka je kašnjenje množača 7 ciklusa i sabirača 4 ciklusa. Pretpostaviti da postoji samo jedan port u registarskoj datoteci za upis vrijednosti u odredišni registar.

Na kraju 17. ciklusa izvršenja sekvence, koji je broj instrukcija koje će imati upisanu vrijednost u history buffer?